

Chapter 7 pt 2: Importing Modules

EEC 3150

Trevecca Nazarene University

Outline

1 User Modules

2 Python Standard Library Modules

User Modules

Defn: module

In general, a **module** is a .py file containing Python definitions and statements

- By this definition, every .py we've written is a module (which is true!) Specifically, this section is concerned with importing the contents of one module into another
- Any functions or variables defined in the first can then be used in the second

User Modules

Importing and dot notation:

- Use the `import` keyword followed by the module name to import it
- Make sure both `.py` files are in the same folder
- We use dot notation to call functions or objects from the module
- This disambiguates them from objects with the same name in the destination module

Example module: `circle.py`

Let's say the following code is saved in a module `circle.py`

```
pi = 3.14159
```

```
def area(radius):  
    return pi*(radius**2)
```

We can import the module into another file `circleTest.py`:

```
import circle
```

```
print(circle.pi)  
print(circle.area(3))
```

Importing multiple modules (pt1):

Dot notation becomes clearly necessary when dealing with multiple modules with functions by the same name:

Module: `circle.py`

```
pi = 3.14159
```

```
def area(radius):  
    return pi*radius**2
```

Module: `sphere.py`

```
pi = 3.14159
```

```
def area(radius):  
    return 4*pi*radius**2
```

Importing multiple modules (pt2):

Let's import both of these modules into one new file:

```
import circle
import sphere

def area(side):
    return side**2

pi = 3
print(pi, circle.pi, sphere.pi)
print(area(1), circle.area(1), sphere.area(1))
```

Different ways to import a module:

- `import module`
 - The basic way
- `import module as md`
 - Give the module a shorter alias
- `from module import submodule`
 - New keyword: `from`
 - Only import what you need from the module
 - No need for dot notation to call the submodule
- `from module import *`
 - Import all submodules without need to use dot notation
 - Dangerous if using multiple objects with the same name
 - Whatever the final version defined/imported is what will be used

Why modules:

Why modules are useful:

- Modules become very useful when your codebase becomes very large
- It lets you compartmentalize different parts of it neatly

Python Standard Library (<https://docs.python.org/3/library/>):

- The majority of Python's codebase is not loaded by default
- Examples: time, math, random, os, sys, string, re, threading

Third-party modules:

- There are also MANY third-party modules (also called libraries or packages) that can be downloaded and installed
- Anaconda comes with all the ones we need
- Examples: NumPy, SciPy, Sci-kit Learn, Keras, NLTK, Pandas, Matplotlib, Seaborn

Python Standard Library Modules

`time:`

The `time` module handles various aspects of getting/setting the time of the system clock and can be used to measure the duration of events in your code

- `time.gmtime([seconds])`
 - No argument: returns the current GMT/UTC as a `struct_time` object
 - With argument: returns the GMT/UTC at the specified number of seconds since January 1, 1970, 00:00:00 (UTC) (called the epoch)
 - `struct_time` objects can be indexed
- `time.localtime([seconds])`
 - Same as `gmtime` but in the local time zone

Python Standard Library Modules

`time:`

The `time` module handles various aspects of getting/setting the time of the system clock and can be used to measure the duration of events in your code

- `time.sleep(seconds)`
 - Pauses program execution for a specified number of seconds
- `time.time()`
 - Returns a float number of seconds since the epoch
 - To measure the duration of a block of code, use two time functions and subtract the difference
 - Measure average times with loops

Python Standard Library Modules

os:

The `os` module handles various operating system and file management processes

- `os.getcwd()`
 - Returns the current working directory (unless changed mid-program, this is the calling module's directory)
- `os.listdir(path='.')`
 - Return a list of files in the specified directory (default is the calling module's directory)
- `os.mkdir(path, ...)`
 - Creates a directory at the specified path
 - Has more arguments
 - Useful for automatically handling scripts that need to write a lot of data files

sys and command line arguments:

The `sys` module provides access to some variables used or maintained by the interpreter and to functions that interact strongly with the interpreter

- `sys.argv`
 - Is a list of command line arguments used when running a script in the console

Most other languages also have an `argc` variable which is the length of `argv`, i.e., the number of command line arguments (Python 3 doesn't)

`pickle`:

The `pickle` module is useful for saving and loading arbitrary, complex data objects (usually aliased `pk`

- `pk.dump(data, fhandle)`
 - Writes an object (`data`) to a file
 - `data` can be literally anything: a 2D list, a dictionary, even an entire deep learning model
- `pk.load(fhandle)`
 - Reads and returns an object from the specified file handle

NOTE: it is **STRONGLY** advised that files be opened in binary mode when using `pickle`, i.e., use `'wb'` and `'rb'` as opposed to simply `'w'` and `'r'` when creating the file handle

Python Standard Library Modules

re and regex:

The `re` and `regex` modules provide methods for *regular expressions* (or `regex`), i.e., tools for finding patterns in text (`re` is in PSL, `regex` is third-party, many similar function but better)

- Patterns:

- A sequence of characters describing what to search for (usually a raw string)
- Raw string: put an `r` before the string (ex: `r"tears in rain"`)

- `re.search(pattern, text)`

- Searches `text` for occurrences of `pattern`
- Returns a `match` object

- `re.findall(pattern, text)`

- Returns a list of all occurrences of `pattern` in `text`
- `overlapped` boolean keyword if using `regex`

re resources: https://www.w3schools.com/python/python_regex.asp